

WEEK 10: Key Distribution Centers, Modular Arithmetic, One-Way Functions & DL

WEEK 10: Key Distribution Centers, Modular Arithmetic, One-Way Functions & DLP

Slide 1: Key Distribution Centers (KDC) – Architecture & Trust Model

Content:

- **Problem:** Symmetric key distribution scales as $O(n^2)$; pairwise key management is infeasible
- **KDC Solution:** Centralized Trusted Third Party (TTP); each entity shares a long-term master key $K_{i,KDC}$ with KDC
- **Kerberos v5 Workflow:**
 1. Client $\rightarrow$ AS: ID_C, ID_{TGS}, TS
 2. AS $\rightarrow$ C: $\{K_{C,TGS}, TGT\}_{K_{C,AS}}, TGT = \{K_{C,TGS}, ID_C, TS, Lifetime\}_{K_{TGS}}$
 3. Client $\rightarrow$ TGS: TGT , Authenticator $\{ID_C, TS\}_{K_{C,TGS}}$
 4. TGS $\rightarrow$ C: $\{K_{C,S}, Ticket_S\}_{K_{C,TGS}}, Ticket_S = \{K_{C,S}, ID_C, TS\}_{K_S}$
- **Security Properties:** Timestamp/replay prevention, mutual auth, forward secrecy (session keys)
- **Limitations:** Single point of failure, clock synchronization dependency, TTP compromise $\rightarrow$ total breach
- **Modern Alternatives:** PKI-based auth, decentralized identity (DID), zero-trust architectures

Speaker Notes:

Emphasize that KDCs solve the *symmetric key distribution* problem but introduce *trust centralization*. Walk through ticket encryption layers. Contrast with public-key infrastructure. Note that modern Kerberos uses AES-CTS-HMAC-SHA1-96 or AES-GCM, not legacy DES.

Suggested Visual:

Sequence diagram of Kerberos AS/TGS flow with encryption layers color-coded.

In-Class Exercise:

Given $K_{C,AS}$, trace how a compromised KDC enables passive decryption of all historical sessions. Discuss mitigations.

Slide 2: Prime Field Arithmetic & Computational Complexity

Content:

- **Definitions:** $\mathbb{Z}_p = \{0, 1, \dots, p-1\}$, $\mathbb{Z}_p^* = \{a \in \mathbb{Z}_p \mid \gcd(a, p) = 1\}$, $|\mathbb{Z}_p^*| = \phi(p) = p-1$

- **Group Structure:** $(\mathbb{Z}_p^*, \cdot)$ is cyclic; $\exists g$ s.t. $\langle g \rangle = \mathbb{Z}_p^*$
- **Modular Inverse:** $a^{-1} \bmod p$ via Extended Euclidean Algorithm in $O(\log^2 p)$
- **Safe Primes:** $p = 2q + 1$ where q is prime; ensures large prime-order subgroups, resists Pohlig-Hellman
- **Exponentiation Algorithms:**
 - Square-and-Multiply: $O(\log e)$ multiplications
 - Montgomery Ladder: constant-time, side-channel resistant
 - Windowed NAF: reduces multiplications at cost of precomputation
- **Complexity:** Modular mult/add: $\tilde{O}(\log p)$ bit ops; exponentiation: $\tilde{O}(\log e \cdot \log p)$

Speaker Notes:

Walk through a small example ($p = 23, g = 5$). Emphasize why safe primes prevent small-subgroup attacks. Note that hardware crypto accelerators use Montgomery reduction.

Common Misconception:

"All primes are equally secure for DLP." $\rightarrow$ False. Smooth $p - 1$ enables Pollard's $p - 1$ attack.

Slide 3: One-Way Functions (OWFs) & Trapdoor Permutations

Content:

- **Formal Definition:** $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$ is OWF if:
 1. $\exists$ PPT algorithm computing $f(x)$
 2. $\forall$ PPT $\mathcal{A}$, $\Pr[\mathcal{A}(f(x)) = x'] \leq \text{negl}(n)$
- **Candidate OWFs:** Modular exponentiation, integer multiplication, cryptographic hashes, lattice functions
- **Trapdoor One-Way Function (TOWF):** $\exists$ efficient inversion with secret td ; e.g., RSA: $f(x) = x^e \bmod n$, $f^{-1}(y) = y^d \bmod n$
- **Relationship to Primitives:**
 - OWF $\Rightarrow$ PRG (Håstad-Impagliazzo-Levin)
 - PRG $\Rightarrow$ PRF (GGM construction)
 - PRF $\Rightarrow$ CPA-secure encryption
- **Open Problem:** Existence of OWFs unproven; equivalent to $P \neq NP$ in average-case complexity

Speaker Notes:

Clarify that OWFs are *assumed*, not proven. Link theoretical constructs to practical schemes. Emphasize that hash functions are compressing OWFs, not permutations.

Discussion Prompt:

If OWFs don't exist, which cryptographic primitives break? (Answer: nearly all of modern crypto.)

Slide 4: Discrete Logarithm Problem (DLP) – Formal Statement & Attacks

Content:

- **DLP:** Given cyclic group G , generator g , and $y = g^x$, find $x \in \{0, \dots, |G| - 1\}$
- **Group Variants:** $\mathbb{Z}_p^*$, $\mathbb{F}_{2^m}^*$, Elliptic curves $E(\mathbb{F}_p)$, Class groups
- **Classical Attacks:**
 - Baby-step Giant-step: $O(\sqrt{|G|})$ time & space
 - Pollard's Rho: $O(\sqrt{|G|})$ time, $O(1)$ space
 - Pohlig-Hellman: exploits smooth $|G|$; reduces to subgroups
 - Index Calculus / NFS-DL: $L_p[1/3, c]$ for $\mathbb{Z}_p^*$
- **Security Parameters:**
 - 2048-bit $p \approx 112$ -bit symmetric security
 - 3072-bit $p \approx 128$ -bit security (NIST SP 800-57)
- **Assumptions:** CDH (Compute g^{ab}), DDH (Distinguish g^{ab} from random)

Speaker Notes:

Compare attack complexities. Stress that DLP hardness is *group-dependent*. Show why $\mathbb{Z}_p^*$ requires larger keys than ECC.

Suggested Visual:

Table mapping group size $\rightarrow$ classical security bits $\rightarrow$ quantum security bits (Shor's $\mathcal{O}((\log p)^3)$).

Slide 5: Security Models & Parameter Generation

Content:

- **Prime Generation:** Miller-Rabin + Lucas test; FIPS 186-5 probabilistic primality
- **Generator Selection:** Verify $g^q \not\equiv 1 \pmod{p}$ for safe primes; avoid trivial/small-order elements
- **Subgroup Confinement:** Validate $1 < y < p$ and $y^q \equiv 1 \pmod{p}$ before use
- **Side-Channel Mitigations:** Constant-time exponentiation, blinding $g^{x+r(p-1)}$, cache-line alignment
- **Standards:** RFC 3526 (MODP groups), RFC 7919 (FFDHE), NIST SP 800-56A Rev 3
- **Implementation Pitfalls:** Weak RNG, small exponents, reused nonces, timing leaks in `powmod`

Speaker Notes:

Bridge theory to practice. Show how validation prevents invalid-curve/subgroup attacks. Mention real CVEs (e.g., CVE-2016-0701, Logjam).

In-Class Exercise:

Given $p = 23$, $g = 2$, $y = 8$, solve DLP by brute force. Then discuss why this fails at 2048 bits.

Slide 6: Week 10 Summary & Self-Assessment

Content:

- KDCs centralize trust to solve symmetric key distribution
- $\mathbb{Z}_p^*$ arithmetic enables efficient, structured computation
- OWFs are unproven but foundational; TOWFs enable PKC
- DLP hardness underpins DH, ElGamal, DSA; security depends on group choice
- Proper parameter generation & constant-time implementation are non-negotiable
- **Next Week:** Building key exchange & signatures on DLP assumptions

Speaker Notes:

Assign reading: Katz & Lindell Ch. 7, NIST SP 800-56A. Preview DH protocol. Remind students to review modular arithmetic before Week 11.



WEEK 11: Diffie-Hellman, ElGamal, Digital Signatures & RSA Signing

Slide 1: Diffie-Hellman Key Exchange – Formal Protocol & Security

Content:

- **Parameters:** Public (p, g) , p large prime, g generator of order $q \mid p - 1$
- **Protocol:**
 1. Alice: $a \xleftarrow{R} \mathbb{Z}_q$, $A = g^a \bmod p$
 2. Bob: $b \xleftarrow{R} \mathbb{Z}_q$, $B = g^b \bmod p$
 3. Shared: $K = B^a = A^b = g^{ab} \bmod p$
- **Security Model:** Passive eavesdropping $\rightarrow$ CDH assumption; Active MITM $\rightarrow$ requires authentication
- **Forward Secrecy:** Ephemeral DH (DHE/ECDHE) ensures past sessions remain secure if long-term keys leak
- **Proof Sketch:** Reduction from CDH adversary to DLP solver; indistinguishability under DDH
- **Vulnerabilities:** Small subgroup attacks, Logjam (downgrade to export ciphers), weak RNG on a, b

Speaker Notes:

Emphasize that DH provides *confidentiality only*. Authentication requires signatures/certificates. Walk through DHE vs static DH.

Suggested Visual:

Protocol timeline with security properties mapped to each message.

Slide 2: ElGamal Encryption – Algorithm & Security Properties

Content:

- **KeyGen:** $x \xleftarrow{R} \mathbb{Z}_q, y = g^x \bmod p$
- **Enc(m):** $k \xleftarrow{R} \mathbb{Z}_q, c_1 = g^k, c_2 = m \cdot y^k \bmod p$
- **Dec(c_1, c_2):** $m = c_2 \cdot (c_1^x)^{-1} \bmod p$
- **Correctness:** $c_2 \cdot c_1^{-x} = m y^k (g^k)^{-x} = m (g^x)^k (g^x)^{-k} = m$
- **Security:** IND-CPA under DDH; NOT IND-CCA2 (malleable: $(c_1, c_2 \cdot \Delta)$ encrypts $m \cdot \Delta$)
- **Homomorphism:** $E(m_1) \cdot E(m_2) = (g^{k_1+k_2}, m_1 m_2 y^{k_1+k_2}) = E(m_1 m_2)$
- **CCA Fix:** Fujisaki-Okamoto transform or hybrid encryption with symmetric AEAD

Speaker Notes:

Contrast with RSA: probabilistic, larger ciphertexts, multiplicative homomorphism. Stress k must be fresh & uniform per encryption.

In-Class Exercise:

Show how malleability breaks IND-CCA2. Propose a CCA-secure variant using HMAC.

Slide 3: Digital Signatures – Formal Framework & Security Game

Content:

- **Syntax:** $\Pi = (\text{KeyGen}, \text{Sign}, \text{Verify})$
 - $(pk, sk) \leftarrow \text{KeyGen}(1^n)$
 - $\sigma \leftarrow \text{Sign}(sk, m)$
 - $b \leftarrow \text{Verify}(pk, m, \sigma) \in \{0, 1\}$
- **Security Model: EUF-CMA**
 1. Challenger gives $\mathcal{A} pk$
 2. $\mathcal{A}$ queries $\text{Sign}(sk, \cdot)$ on $m_1, \dots, m_q$
 3. $\mathcal{A}$ outputs (m^*, σ^*) s.t. $m^* \notin \{m_i\}$ and $\text{Verify}(pk, m^*, \sigma^*) = 1$
 4. Advantage $\text{Adv}_{\Pi, \mathcal{A}}^{\text{EUF-CMA}}(n) = \Pr[\text{forge}]$
- **Hash-then-Sign:** $\sigma = \text{Sign}_{sk}(\text{H}(m))$; requires H collision-resistant
- **Random Oracle Model:** Security proofs often assume H is ideal; standard model proofs exist but less efficient

Speaker Notes:

EUF-CMA is the industry standard. Emphasize that signing raw messages is insecure. Explain why collision resistance matters.

Common Misconception:

"Signatures encrypt the message." $\rightarrow$ False. They prove authenticity & integrity, not confidentiality.

Slide 4: RSA Signatures – Padding & Security Proofs

Content:

- **KeyGen:** p, q primes, $n = pq$, $\phi(n) = (p-1)(q-1)$, pick $e, d \equiv e^{-1} \pmod{\phi(n)}$
- **Naive Sign:** $\sigma = H(m)^d \pmod{n} \rightarrow$ Vulnerable to homomorphic forgery & root extraction
- **PKCS#1 v1.5:** Deterministic padding: $0x00||0x01||PS||0x00||ASN.1(H)||H$
 - Bleichenbacher'96: Chosen-message forgery via padding oracle
- **PSS (Probabilistic Signature Scheme):**
 - Salt $r \xleftarrow{R} \{0, 1\}^{hLen}$, $DB = \text{MGF1}(h, pLen), maskedDB, EM = 0x00||maskedDB||h||0x01$
 - $\sigma = EM^d \pmod{n}$
 - Provable EUF-CMA under RSA assumption + random oracle
- **Standards:** RFC 8017 (PKCS#1), FIPS 186-5 mandates PSS for new apps

Speaker Notes:

Show v1.5 vs PSS structure. Explain why probabilistic padding thwarts deterministic forgeries. Note that RSA sign is slower than verify.

Suggested Visual:

Byte-level layout of PKCS#1 v1.5 vs PSS.

Slide 5: Signature Implementation Security & Real-World Failures

Content:

- **Nonce Reuse (DSA/ECDSA):** k reused $\rightarrow s_1^{-1}(H(m_1) + dr) \equiv s_2^{-1}(H(m_2) + dr) \pmod{q} \rightarrow d$ recoverable
- **Hash Collisions:** SHA-1 broken (SHAttered, 2017); deprecated in code signing & TLS
- **Side Channels:** Timing on modular exponentiation, cache attacks on table lookups
- **Mitigations:** RFC 6979 (deterministic k), constant-time libraries (libsodium, BoringSSL), hardware RNG
- **CVE Examples:**
 - CVE-2019-17006 (Firefox ECDSA nonce reuse)
 - CVE-2020-0601 (Windows CryptoAPI curve validation bypass)
- **Best Practice:** Use Ed25519/Ed448 or RSA-PSS; avoid custom padding; validate all inputs

Speaker Notes:

Highlight that most signature breaks are implementation flaws, not mathematical. Reference Sony PS3 ECDSA leak.

Discussion Prompt:

Why is deterministic nonce generation (RFC 6979) safer than hardware RNG in constrained

Slide 6: Week 11 Summary & Lab Outline

Content:

- DH enables forward-secure key exchange over insecure channels
- ElGamal provides probabilistic, homomorphic encryption (IND-CPA)
- EUF-CMA is the gold standard for signature security
- RSA signing requires secure padding (PSS preferred over v1.5)
- Nonce management & hash strength are critical in practice
- **Lab:** Implement ElGamal encryption & RSA-PSS signing in Python; test malleability & nonce reuse attacks

Speaker Notes:

Assign lab with starter code. Emphasize testing against known vectors. Preview ECC as a more efficient alternative.



WEEK 12: Elliptic Curve Cryptography, Pairings & ECDSA

Slide 1: Elliptic Curve Group Law & Coordinate Systems

Content:

- **Weierstrass Equation:** $y^2 = x^3 + ax + b \bmod p$, $\Delta = -16(4a^3 + 27b^2) \not\equiv 0$
- **Point Addition (Affine):**
 - $P + Q$: $\lambda = (y_Q - y_P)(x_Q - x_P)^{-1}$, $x_R = \lambda^2 - x_P - x_Q$, $y_R = \lambda(x_P - x_R) - y_P$
 - $2P$: $\lambda = (3x_P^2 + a)(2y_P)^{-1}$
- **Coordinate Systems:**
 - Affine: 1 inversion per add/double (expensive)
 - Jacobian: (X, Y, Z) , $x = X/Z^2$, $y = Y/Z^3$, avoids inversions
 - Projective/Extended Edwards: constant-time, side-channel resistant
- **Scalar Multiplication:** kP via double-and-add, windowed NAF, Montgomery ladder
- **Group Properties:** Abelian, identity $\mathcal{O}$, order $\#E(\mathbb{F}_p) = p + 1 - t$, Hasse bound $|t| \leq 2\sqrt{p}$

Speaker Notes:

Explain why inversions are costly in $\mathbb{F}_p$. Show how Jacobian coordinates trade space for speed. Emphasize constant-time scalar multiplication.

Suggested Visual:

Geometric chord-tangent addition over $\mathbb{R}$, then map to $\mathbb{F}_p$.

Slide 2: ECDLP Hardness & Curve Security Criteria

Content:

- **ECDLP:** Given $P, Q = kP$, find k
- **Best Classical Attack:** Pollard's Rho $\mathcal{O}(\sqrt{n})$ time, $\mathcal{O}(1)$ space
- **No Sub-Exponential Attacks:** Index calculus doesn't generalize to generic elliptic curves
- **Security Equivalence:** 256-bit ECC $\approx$ 3072-bit RSA $\approx$ 128-bit symmetric
- **Safe Curve Criteria (Bernstein-Lange):**
 - Large prime order subgroup
 - Complete addition formulas (no exceptional cases)
 - Twist security (resist invalid-curve attacks)
 - Verifiably random parameters (avoid backdoors)
- **Standards:** NIST P-256 (controversial seed), Curve25519, secp256k1, Ed25519

Speaker Notes:

Discuss NIST curve seed controversy. Contrast with Curve25519's transparent generation. Warn against custom curves.

In-Class Exercise:

Given $\#E = 2^{256} - 2^{224} + 2^{192} + 2^{96} - 1$, estimate security level. Compare to RSA key size.

Slide 3: Bilinear Pairings – Definition, Construction & Applications

Content:

- **Bilinear Map:** $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$
 - Bilinear: $e(aP, bQ) = e(P, Q)^{ab}$
 - Non-degenerate: $e(P, Q) \neq 1$ for generators
 - Efficiently computable
- **Constructions:** Weil, Tate, Ate pairings; Miller's algorithm $\mathcal{O}(\log p)$ field ops
- **Embedding Degree k :** Smallest k s.t. $r \mid p^k - 1$; controls MOV attack resistance
- **Pairing-Friendly Curves:** BN (Barreto-Naehrig), BLS, KSS; trade security for efficiency
- **Applications:**
 - Identity-Based Encryption (Boneh-Franklin)
 - BLS signatures (short, aggregatable)
 - zk-SNARKs (Groth16, Plonk)
 - Threshold cryptography, ring signatures

- **Security:** BDHP, SXDH, subgroup hiding; embedding degree ≥ 8 recommended

Speaker Notes:

Keep pairings conceptual. Note that MOV reduces ECDLP to DLP in $\mathbb{F}_{p^k}^*$ if k is small. Emphasize that pairings enable "cryptographic glue."

Suggested Visual:

Group mapping diagram: $\mathbb{G}_1 \times \mathbb{G}_2 \xrightarrow{e} \mathbb{G}_T$ with bilinearity arrows.

Slide 4: ECDSA Algorithm – Specification & Verification Logic

Content:

- **Setup:** Curve $E/\mathbb{F}_p$, base point G , order n , cofactor h
- **KeyGen:** $d \xleftarrow{R} [1, n-1]$, $Q = dG$
- **Sign(m):**
 1. $k \xleftarrow{R} [1, n-1]$
 2. $R = kG = (x_R, y_R)$, $r = x_R \bmod n$
 3. $s = k^{-1}(\text{H}(m) + dr) \bmod n$
 4. Output (r, s)
- **Verify(m, r, s):**
 1. Check $1 \leq r, s < n$
 2. $w = s^{-1} \bmod n$, $u_1 = \text{H}(m)w \bmod n$, $u_2 = rw \bmod n$
 3. $R' = u_1G + u_2Q$, accept if $R'_x \equiv r \bmod n$
- **Critical:** k must be uniform & secret; reuse leaks d via linear algebra
- **Deterministic ECDSA:** RFC 6979: $k = \text{HMAC-SHA256}(d || \text{H}(m))$

Speaker Notes:

Walk through signing/verification step-by-step. Show how k reuse yields d . Explain why deterministic k is safer in constrained devices.

Common Misconception:

"ECDSA is just DSA on curves." $\rightarrow$ False. Curve structure enables different attacks & optimizations.

Slide 5: EdDSA, Implementation Security & Modern Practices

Content:

- **EdDSA (Edwards-curve DSA):**
 - Twisted Edwards: $by^2 + x^2 = 1 + dx^2y^2$
 - Complete addition formulas, no exceptional cases
 - Deterministic nonce via $\text{HMAC}(sk || \text{H}(m))$

- Cofactor clearing: $Q = [h] \cdot \text{Hash}(m)$

- **Ed25519/Ed448:**

- Curve: Curve25519/Curve448
- Hash: SHA-512/SHAKE256
- Security: EUF-CMA, side-channel resistant, fast verification

- **Implementation Pitfalls:**

- Invalid point rejection
- Timing leaks in scalar multiplication
- Poor RNG for key generation

- **Standards:** FIPS 186-5, RFC 8032, NIST SP 800-186

- **Recommendation:** Prefer EdDSA over ECDSA for new deployments

Speaker Notes:

Show why complete formulas prevent fault attacks. Note that EdDSA eliminates the k reuse vulnerability by design.

In-Class Exercise:

Compare ECDSA vs EdDSA: key size, signature size, verification speed, nonce management.

Slide 6: Week 12 Summary & Research Directions

Content:

- ECC provides equivalent security to RSA/DH with smaller keys & faster ops
- ECDLP hardness underpins ECDSA, ECDH, pairing-based crypto
- Pairings enable advanced protocols but require careful parameterization
- ECDSA requires strict nonce management; EdDSA fixes this by design
- **Open Problems:** Verifiable curve generation, side-channel resistant pairing implementations, post-quantum curve alternatives
- **Lab:** Implement ECDSA signing/verification; test nonce reuse attack; benchmark vs Ed25519

Speaker Notes:

Assign lab with curve validation checks. Preview PQC threats to ECC. Recommend reading: Bernstein et al. "SafeCurves".

WEEK 13: Post-Quantum Cryptography, Lattices, Isogenies & ID Protocols

Slide 1: Quantum Threat & PQC Standardization Landscape

Content:

- **Shor's Algorithm:** Solves factoring, DLP, ECDLP in $\mathcal{O}((\log N)^3)$ time on fault-tolerant quantum computer
- **Grover's Algorithm:** Speeds up brute-force search to $\mathcal{O}(\sqrt{N})$; doubles symmetric key length requirement
- **NIST PQC Competition (2016–2024):**
 - KEM: CRYSTALS-Kyber (FIPS 203)
 - Signatures: CRYSTALS-Dilithium (FIPS 204), Falcon (FIPS 205), SPHINCS+
- **Security Levels (1–5):** Mapped to AES-128/192/256 & SHA-256/384/512
- **Deployment Strategy:** Hybrid classical+PQC during transition; certificate chaining; protocol agility
- **Timeline:** CNSA 2.0 mandates PQC by 2030; TLS 1.3 PQC extensions in draft

Speaker Notes:

Clarify that "post-quantum" $\neq$ quantum computing. Emphasize that PQC is already being deployed in browsers, SSH, and firmware.

Suggested Visual:

Timeline of NIST PQC rounds $\rightarrow$ FIPS publication $\rightarrow$ industry adoption.

Slide 2: Lattice-Based Cryptography – Foundations & Schemes

Content:

- **Lattice:** $\mathcal{L} = \{\sum x_i b_i \mid x_i \in \mathbb{Z}\}$, basis $B \in \mathbb{R}^{n \times n}$
- **Hard Problems:** SVP (Shortest Vector), CVP, GapSVP, UniqueSVP
- **Learning With Errors (LWE):**
 - Given $(A, b = As + e \bmod q)$, recover s
 - $A \in \mathbb{Z}_q^{m \times n}$, $e \leftarrow \chi$ (discrete Gaussian)
 - Worst-case to average-case reduction (Regev 2005)
- **Ring/Module-LWE:** Algebraic structure improves efficiency; used in Kyber/Dilithium
- **Kyber (KEM):** IND-CCA2 secure, FO transform, public key ~1.2KB, ciphertext ~1KB
- **Dilithium (Sign):** EUF-CMA, Fiat-Shamir with aborts, signature ~2.4KB
- **Advantages:** Parallelizable, simple arithmetic, strong security reductions
- **Challenges:** Large key/ciphertext sizes, side-channel resistance, constant-time implementations

Speaker Notes:

Explain LWE intuitively: noisy linear equations. Note that Module-LWE balances security & efficiency. Mention that lattice crypto is mathematically clean.

In-Class Exercise:

Given A, b , show why recovering s is hard without knowing error distribution.

Slide 3: Isogeny-Based Cryptography – Promise & Break

Content:

- **Isogeny:** Rational map $\phi : E_1 \rightarrow E_2$ preserving group structure; degree $\deg(\phi)$
- **SIDH/SIKE Protocol:** Key exchange using secret isogeny paths; compact keys (~330 bytes)
- **Security Assumption:** Hardness of computing isogeny between two elliptic curves given auxiliary torsion points
- **2022 Break (Castryck-Decru):**
 - Exploits Kani's theorem & Weil pairing
 - Recovers secret isogeny using torsion point structure
 - Polynomial-time attack on SIKEp434/503/610
- **Current Status:** SIKE withdrawn from NIST; research continues in SQISign, CSIDH (supersingular isogeny DH)
- **Why Study Isogenies?** Mathematically rich, compact keys, potential for threshold/IBE variants
- **Lesson:** Cryptographic assumptions must withstand decades of cryptanalysis

Speaker Notes:

Be explicit about the break. Use it as a case study in assumption validation. Note that isogenies remain active research but lack standardized security margins.

Discussion Prompt:

Does the SIKE break invalidate all isogeny crypto, or just SIDH-based constructions? Why?

Slide 4: Identification & Login Protocols – Sigma Protocols & Modern Auth

Content:

- **Sigma Protocol (Σ):** 3-move honest-verifier ZK proof
 1. Commitment: $t = g^r$
 2. Challenge: $c \xleftarrow{R} \{0, 1\}^\lambda$
 3. Response: $s = r + cx \bmod q$
 - Completeness, special soundness, honest-verifier ZK
- **Fiat-Shamir Transform:** $\Sigma \rightarrow \text{NIZK}$ signature via $c = H(t||m)$
- **Schnorr ID/Sign:** EUF-CMA under DL + random oracle; basis for EdDSA
- **Password Auth:** PBKDF2, Argon2id, SCRYPT; memory-hard, GPU/ASIC resistant
- **FIDO2/WebAuthn:** Public-key auth, platform authenticators, phishing resistance, hardware-backed keys
- **Security Models:** eCK, BR, UC framework; resistance to KCI, weak forward secrecy, replay

Speaker Notes:

Show how Schnorr ID becomes Schnorr signatures via Fiat-Shamir. Contrast password hashing with cryptographic ID. Note industry shift to passkeys.

Suggested Visual:

Sigma protocol message flow with security properties annotated.

Slide 5: Security Considerations & PQC Deployment

Content:

- **Hybrid Crypto:** Classical + PQC KEM during transition; mitigates algorithmic risk
- **Parameter Selection:** NIST levels 1–5, bandwidth/latency trade-offs, hardware acceleration
- **Side Channels:** Lattice decoding timing, fault attacks on Dilithium, countermeasures (masking, shuffling)
- **Standardization:** FIPS 203/204/205 (2024), IETF TLS PQC drafts, X.509 PQC extensions
- **Migration Path:** Protocol agility, certificate chaining, fallback mechanisms, inventory assessment
- **Regulatory:** CNSA 2.0, ETSI QKD standards, ISO/IEC JTC 1/SC 27
- **Implementation Libraries:** liboqs, Open Quantum Safe, Bouncy Castle PQC, libsodium (Ed25519)

Speaker Notes:

Stress that PQC deployment is a systems engineering problem. Hybrid schemes are mandatory for high-security apps. Mention real-world pilots (Chrome, Cloudflare, AWS).

In-Class Exercise:

Design a hybrid TLS handshake combining ECDHE + Kyber. List security properties preserved.

Slide 6: Week 13 Summary & Research Directions

Content:

- Quantum computing threatens RSA/ECC/DH; PQC provides classical alternatives
- Lattice crypto leads standardization; strong reductions, efficient implementations
- Isogenies broken but mathematically valuable; caution in assumption design
- Secure login relies on challenge-response, password hardening, hardware auth
- **Open Problems:** Efficient PQC ZK, quantum-safe MPC, standardized threshold PQC, usable security
- **Reading:** NIST IR 8413, "Post-Quantum Cryptography" (Bernstein et al.), IETF PQC drafts

Speaker Notes:

Assign reading. Preview ZKP & MPC. Encourage students to explore liboqs examples.

🕒 WEEK 14: Zero-Knowledge Proofs, Authenticated Key Exchange & MPC

Slide 1: Zero-Knowledge Proofs – Formal Foundations

Content:

- **Definition:** Prover convinces Verifier statement $x \in L$ without revealing witness w
- **Properties:**
 - Completeness: $\Pr[\text{accept} | x \in L] = 1$
 - Soundness: $\Pr[\text{accept} | x \notin L] \leq \epsilon$
 - Zero-Knowledge: $\exists$ simulator $\mathcal{S}$ s.t. $\mathcal{S}(x) \approx_c \text{view}_V$
- **Models:** Computational vs Statistical vs Perfect ZK; Interactive vs NIZK
- **Classic Examples:**
 - Graph 3-Coloring (NP-complete)
 - Discrete Log (Schnorr)
 - Quadratic Residuosity
- **Random Oracle vs CRS:** Fiat-Shamir needs RO; transparent SNARKs need no setup
- **Applications:** Anonymous credentials, privacy coins, verifiable computation, regulatory compliance

Speaker Notes:

Use Ali Baba cave analogy. Distinguish simulation-based ZK from heuristic "hiding." Note that statistical ZK requires more rounds/assumptions.

Suggested Visual:

Simulator diagram: Ideal world vs Real world indistinguishability.

Slide 2: Modern ZK Systems – SNARKs, STARKs & Efficiency

Content:

- **zk-SNARKs:** Succinct Non-Interactive Arguments of Knowledge
 - Pinocchio, Groth16, Plonk
 - Trusted setup (CRS), $O(1)$ verification, ~200B proofs
 - Arithmetic circuits $\rightarrow$ R1CS $\rightarrow$ QAP $\rightarrow$ polynomial commitments
- **zk-STARKs:** Scalable Transparent ARguments of Knowledge
 - No trusted setup, post-quantum (hash-based), larger proofs (~100KB)
 - FRI protocol, hash functions, collision resistance
- **Bulletproofs:** Logarithmic proof size, no setup, used in Monero

- **Efficiency Trade-offs:** Setup trust vs proof size vs verification speed vs quantum resistance
- **Applications:** Zcash, Ethereum rollups, private voting, verifiable ML inference
- **Open Problems:** Transparent SNARKs, efficient recursive proofs, hardware acceleration

Speaker Notes:

Contrast SNARKs (fast, trusted) vs STARKs (transparent, larger). Explain why R1CS is standard for circuit compilation. Note that trusted setup is a centralization risk.

In-Class Exercise:

Map a simple constraint (e.g., $x \cdot y = z$) to R1CS format.

Slide 3: Authenticated Key Exchange (AKE) – Models & Protocols

Content:

- **Security Models:** BR (Bellare-Rogaway), eCK (extended Canetti-Krawczyk), CK+
 - Captures: forward secrecy, KCI, weak key compromise, session key leakage
- **SIGMA Protocol:**
 - $A \rightarrow B : g^a, \text{Sign}_A(g^a, g^b)$
 - $B \rightarrow A : g^b, \text{Sign}_B(g^a, g^b)$
 - Basis for IKEv2, TLS 1.3
- **HMQRV:** Flawed security proof; fixed by variants; vulnerable to unknown key-share
- **Noise Protocol:** Modular, state machine, widely used (WhatsApp, Signal, WireGuard)
- **TLS 1.3 AKE:** Mandatory forward secrecy, 1-RTT, PSK resumption, key confirmation
- **Threats:** Replay, downgrade, KCI, reflection, weak RNG, certificate validation bypass

Speaker Notes:

Emphasize that AKE $\neq$ key exchange. Authentication prevents MITM. TLS 1.3 is the modern reference. Noise is developer-friendly.

Suggested Visual:

Protocol comparison matrix: security properties vs round complexity vs deployment.

Slide 4: Secure Multi-Party Computation (MPC) – Theory & Practice

Content:

- **Ideal/Real Paradigm:** Protocol Π emulates ideal functionality $\mathcal{F}$; adversary $\mathcal{A}$ in real world $\approx$ simulator $\mathcal{S}$ in ideal world
- **Adversary Models:** Semi-honest (honest-but-curious), malicious, static vs adaptive, synchronous vs asynchronous
- **Core Techniques:**

- Secret Sharing: Shamir (t, n) , additive
- Garbled Circuits: Yao's protocol, OT-based, malicious security via cut-and-choose
- GMW: Boolean/arithmetic circuits, OT-based
- SPDZ: Preprocessing + online phase, MACs, malicious security
- **Complexity:** Communication $\mathcal{O}(n^2)$ rounds, computation $\mathcal{O}(|C|)$ gates, latency-bound
- **Applications:** Threshold signatures (FROST), private analytics, auctions, privacy-preserving ML
- **Frameworks:** MP-SPDZ, ABY, Concrete, TFHE (for FHE+MPC hybrids)

Speaker Notes:

Clarify that MPC is theoretically universal but practically constrained by communication. Note that preprocessing reduces online latency. Mention real-world deployments (JPMorgan, Cloudflare, Uniswap).

In-Class Exercise:

Design a 2-party MPC for computing $\max(x, y)$ using garbled circuits. Estimate OT count.

Slide 5: Cross-Protocol Synergies & Emerging Applications

Content:

- **Threshold Cryptography:** FROST (signing), DKG (distributed key gen), reduces single point of failure
- **Privacy-Preserving ML:** Secure inference, training on encrypted data, ZK-proof of model correctness
- **Blockchain Integration:** ZK rollups, anonymous transactions, cross-chain MPC bridges
- **Hybrid Architectures:** ZK + MPC (verifiable computation), FHE + MPC (secure aggregation), AKE + PQC
- **Regulatory Drivers:** GDPR/CCPA data minimization, right to be forgotten, auditability without exposure
- **Standardization:** IETF MPC drafts, W3C Verifiable Credentials, NIST IR 8413, ISO/IEC 23005
- **Open Challenges:** Efficient malicious security, post-quantum ZK, usable privacy tech, formal verification

Speaker Notes:

Show how primitives converge in modern privacy tech. Note that regulatory pressure accelerates adoption. Mention that MPC is moving from academia to production.

Discussion Prompt:

Why is threshold signing preferred over HSM-based signing in decentralized systems?

Slide 6: Week 14 Summary & Lab/Assessment

Content:

- ZKP enables verifiable privacy without trust
- AKE combines secrecy & authentication with formal security models
- MPC allows collaborative computation with input privacy
- Trade-offs (communication, setup, trust) dictate real-world feasibility
- **Lab:** Implement a 2-party garbled circuit for secure comparison; benchmark vs plaintext
- **Assessment:** Protocol security analysis exercise, ZK proof sketch, AKE threat model

Speaker Notes:

Assign lab with starter code. Emphasize threat modeling. Preview TLS internals & advanced topics.



WEEK 15: TLS Session Setup & Advanced Cryptography Topics

Slide 1: TLS Architecture & Cryptographic Suites

Content:

- **Protocol Layers:**
 - Record: Framing, AEAD encryption, sequence numbers, replay protection
 - Handshake: Negotiation, authentication, key exchange
 - Alert: Error/notification codes
 - ChangeCipherSpec: Legacy (deprecated in TLS 1.3)
- **Cipher Suite Naming:** `TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384`
 - KEX: ECDHE, Auth: RSA, Bulk: AES-256-GCM, MAC/PRF: SHA384
- **Certificate Chain:** Root → Intermediate → Leaf; validation via PKI, OCSP stapling, CT logs
- **Security Properties:** Confidentiality, integrity, forward secrecy, server/client auth, downgrade resistance
- **Vulnerabilities:** POODLE, BEAST, CRIME, BREACH, Lucky13, Heartbleed (historical)
- **Modern Defaults:** AEAD only, mandatory forward secrecy, RSA key transport removed, 0-RTT optional

Speaker Notes:

Walk through layer responsibilities. Explain why AEAD replaces MAC+encrypt. Note that TLS 1.3 removed legacy ciphers.

Suggested Visual:

TLS protocol stack with layer functions mapped.

Slide 2: TLS 1.3 Handshake & Key Schedule

Content:

- **1-RTT Flow:**

1. ClientHello: supported groups, cipher suites, key share, PSK ID
2. ServerHello: selected suite, key share, cookie
3. EncryptedExtensions, Certificate, CertificateVerify, Finished

- **Key Derivation (HKDF):**

- Extract: $PRK = \text{HKDF-Extract}(salt, IKM)$
- Expand: $OKM = \text{HKDF-Expand}(PRK, info, L)$
- Traffic keys: separate handshake/app keys, forward secrecy per direction

- **0-RTT (PSK):** Early data, replay risk, idempotent-only, limited security

- **Security Model:** Proven under eCK with authenticated key confirmation; formal verification (CryptoVerif, Tamarin)

- **Implementation:** OpenSSL 3.x, BoringSSL, rustls, GnuTLS; constant-time AEAD, session resumption

Speaker Notes:

Explain why TLS 1.3 is simpler & more secure. Highlight HKDF's role in key separation. Note that 0-RTT is controversial but useful.

In-Class Exercise:

Trace key derivation from shared secret to client/server traffic keys using HKDF steps.

Slide 3: Advanced Cryptographic Primitives

Content:

- **Fully Homomorphic Encryption (FHE):** Compute on ciphertexts
 - Schemes: BFV (integer), CKKS (approximate), TFHE (boolean)
 - Bootstrapping: Refresh noise, enables unlimited depth
 - Applications: Private ML, secure cloud computation, regulatory compliance
- **Functional Encryption (FE):** Decrypt specific functions $f(m)$
 - Attribute-Based Encryption (ABE), Inner Product FE
 - Use cases: Fine-grained access control, encrypted search
- **Quantum Key Distribution (QKD):** Physics-based security
 - BB84 protocol, no-cloning theorem, intercept-resend detection
 - Limitations: Distance, trusted nodes, not a PKC replacement
- **Blockchain Cryptography:** Merkle trees, ECDSA/EdDSA, ZK-SNARKs, threshold signing

- **AI in Crypto:** Automated cryptanalysis, side-channel detection, formal verification assistants

Speaker Notes:

Position each as active research. Note that FHE is becoming practical for niche use cases. QKD is complementary, not replacement.

Suggested Visual:

Primitive comparison matrix: security assumptions, efficiency, quantum resistance, deployment status.

Slide 4: Cryptanalysis & Security Engineering

Content:

- **Side-Channel Attacks:**
 - Timing: RSA decryption, ECDSA scalar multiplication
 - Power/EM: Smart cards, IoT
 - Cache: AES table lookups, branch prediction
 - Fault: Voltage glitching, laser injection
- **Countermeasures:** Blinding, masking, constant-time code, dummy operations, hardware randomization
- **Formal Verification:**
 - Tools: Tamarin, ProVerif, CryptoVerif
 - Verified libs: HACL*, EverCrypt, s2n-tls
 - Reduces human error, ensures protocol compliance
- **Vulnerability Management:** CVE process, coordinated disclosure, patch deployment, legacy support
- **Best Practices:** Use audited libraries, disable weak ciphers, monitor standards, implement defense-in-depth

Speaker Notes:

Emphasize that most breaks are implementation flaws, not mathematical. Note that formal verification is becoming mainstream.

Discussion Prompt:

Why is constant-time programming harder than it sounds? Give examples of compiler optimizations that break it.

Slide 5: Emerging Standards & Future Directions

Content:

- **PQC Integration:** TLS 1.3 hybrid KEM, SSH, IKEv2, X.509 extensions, certificate chaining

- **Regulatory Landscape:** CNSA 2.0, ETSI QKD, ISO/IEC standards, NIST roadmap, GDPR/CCPA compliance
- **Usable Security:** Passkeys, WebAuthn, privacy-by-design, developer-friendly APIs
- **Privacy-Enhancing Technologies (PETs):** ZK, MPC, FHE, differential privacy, homomorphic hashing
- **Open Research Problems:**
 - Efficient post-quantum ZK & MPC
 - Standardized threshold PQC
 - Hardware-accelerated crypto (RISC-V extensions, FPGA)
 - AI-assisted cryptanalysis & verification
- **Career Paths:** Cryptographer, security engineer, protocol developer, auditor, researcher

Speaker Notes:

Highlight that crypto deployment is now a systems engineering problem. Encourage students to contribute to open-source libs or standards.

Suggested Visual:

Roadmap: Academic research → standardization → implementation → deployment → regulation.

Slide 6: Course Synthesis & Assessment

Content:

- **Core Competencies:**
 - Mathematical foundations (groups, fields, lattices)
 - Protocol design & security models (EUF-CMA, eCK, simulation)
 - Implementation rigor (constant-time, RNG, validation)
 - Standards & compliance (FIPS, RFCs, NIST)
- **Exam Topics:** DLP vs ECDLP, ECDSA nonce math, TLS 1.3 key schedule, ZK properties, PQC categories
- **Final Project:** Implement & analyze a cryptographic protocol; formal security sketch; performance benchmark
- **Resources:** Katz & Lindell, Boneh & Shoup, IACR ePrint, liboqs, CryptoPals challenges
- **Q&A & Office Hours:** Schedule, grading rubric, career advising, open research opportunities

Speaker Notes:

Use for interactive review. Pose 2–3 conceptual questions. Direct students to past papers & recommended readings. Encourage hands-on practice.

Component	Recommendation
Pacing	50 min lecture + 10 min Q&A per session. Slides 1–3: theory/math, 4–5: implementation/security, 6: wrap/exercise
Math Rendering	Use LaTeX Beamer or PowerPoint Equation Editor. All equations are compatible.
Labs	Provide starter code in Python/C/Rust. Use <code>liboqs</code> , <code>cryptography</code> , <code>libsodium</code> . Test against known vectors.
Assessments	Mix theoretical proofs, security game analysis, implementation debugging, protocol design.
Updates	Verify NIST FIPS 203/204/205, IETF TLS drafts, and CVE databases before delivery.
Accessibility	Provide slide PDFs, code templates, math cheat sheets, and recorded walkthroughs.